POMPS Presentation: 1.2.83

Subhodeep Chakraborty
ee25btech11055,
IIT Hyderabad.

April 1, 2026

1. Problem Statement
2. Limit Analysis
3. Reverse Newton-Raphson
4. Fixed-Point Theorem
5. Python Codes
6. Simulation Verification

Problem Statement

Question 1.2.83 (CS 2008)

The Newton-Raphson iteration $x_{n+1} = \frac{1}{2}\left(x_n + \frac{R}{x_n}\right)$ can be used to compute the

a) square of R
b) reciprocal of R
c) square root of R
d) logarithm of R

Method 1: Steady-State / Limit Analysis

If the sequence x_n converges to a limit L as $n \to \infty$, then we can state that the steady-state holds:

$$\lim_{n\to\infty} x_{n+1} = \lim_{n\to\infty} x_n = L \tag{2.1}$$

Substituting this into the given iteration equation:

$$L = \frac{1}{2}\left(L + \frac{R}{L}\right) \tag{2.2}$$

$$2L = L + \frac{R}{L} \tag{2.3}$$

$$L = \frac{R}{L} \tag{2.4}$$

$$L^2 = R \implies L = \sqrt{R} \tag{2.5}$$

Thus, the iteration computes the square root of R.
Answer: (c)

Method 2: Reverse-Engineering Newton-Raphson

The generalized Newton-Raphson formula for finding roots of $f(x) = 0$ is:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)} \tag{3.1}$$

On manipulating the given expression into this standard form:

$$x_{n+1} = \frac{x_n}{2} + \frac{R}{2x_n} \tag{3.2}$$

$$x_{n+1} = x_n - x_n + \frac{x_n}{2} + \frac{R}{2x_n} \tag{3.3}$$

$$x_{n+1} = x_n - \left(\frac{x_n}{2} - \frac{R}{2x_n}\right) \tag{3.4}$$

$$x_{n+1} = x_n - \frac{x_n^2 - R}{2x_n} \tag{3.5}$$

Method 2: Reverse-Engineering Newton-Raphson

Comparing with the standard N-R formula:

$$\frac{f(x_n)}{f'(x_n)} = \frac{x_n^2 - R}{2x_n} \tag{3.6}$$

By observation, if we set the numerator to be the function and the denominator to be its derivative:

$$f(x) = x^2 - R \tag{3.7}$$

$$f'(x) = 2x \tag{3.8}$$

The Newton-Raphson method finds the roots of $f(x) = 0$.

$$x^2 - R = 0 \implies x = \sqrt{R} \tag{3.9}$$

This confirms it evaluates the square root of R.

Method 3: Fixed-Point Theorem & Cobweb Plot

The given sequence $x_{n+1} = \frac{1}{2}\left(x_n + \frac{R}{x_n}\right)$ can be written as a fixed-point iteration function:

$$x_{n+1} = g(x_n) \tag{4.1}$$

where $g(x) = \frac{1}{2}\left(x + \frac{R}{x}\right)$.

For a fixed-point iteration to converge to a limit L, two things must be true:

- **The Fixed Point:** The limit must satisfy $L = g(L)$. Graphically, this is where the curve $y = g(x)$ intersects the identity line $y = x$.

Method 3: Fixed-Point Theorem & Cobweb Plot

- **The Convergence Condition (Stability):** The derivative of $g(x)$ at the root must have an absolute value less than 1, meaning $|g'(L)| < 1$.

Let's find the derivative of $g(x)$:

$$g'(x) = \frac{1}{2}\left(1 - \frac{R}{x^2}\right) \tag{4.2}$$

At the root $x = \sqrt{R}$, the derivative is:

$$g'\left(\sqrt{R}\right) = \frac{1}{2}\left(1 - \frac{R}{(\sqrt{R})^2}\right) = 0 \tag{4.3}$$

Because the derivative is exactly zero, this rigorously proves not just convergence, but **quadratic (superlinear) convergence**.

Proof of Quadratic Convergence

With the derived condition $g'(L) = 0$, we analyse the error $e_n = x_n - L$.

The error at the next step is:

$$e_{n+1} = x_{n+1} - L = g(x_n) - L \tag{4.4}$$

$$e_{n+1} = g(L + e_n) - L \tag{4.5}$$

Expanding $g(L + e_n)$ using a Taylor series around the root L:

$$g(L + e_n) = g(L) + g'(L)e_n + \frac{g''(c)}{2!}e_n^2 \tag{4.6}$$

where c is a value between L and x_n.

Proof of Quadratic Convergence

We apply our known conditions at the fixed point: $g(L) = L$ and the derivative condition $g'(L) = 0$.

Substituting these into the Taylor expansion:

$$g(L + e_n) = L + (0)e_n + \frac{g''(c)}{2}e_n^2 \tag{4.7}$$

$$g(L + e_n) = L + \frac{g''(c)}{2}e_n^2 \tag{4.8}$$

Substituting back into our error equation $e_{n+1} = g(L + e_n) - L$:

$$e_{n+1} = \frac{g''(c)}{2}e_n^2 \implies |e_{n+1}| = \left|\frac{g''(c)}{2}\right| |e_n|^2 \tag{4.9}$$

Since $|e_{n+1}| \propto |e_n|^2$, the error is squared at each step, proving **quadratic convergence**.

Python Implementation I

```python
import numpy as np
import matplotlib.pyplot as plt
import sympy as sp
from scipy import optimize
import os

# ---------------------------------------------------------
# Setup & Initializations
# ---------------------------------------------------------
output_dir = '../figs/'
os.makedirs(output_dir, exist_ok=True)

R_val = 25.0
true_root_pos = np.sqrt(R_val)
true_root_neg = -np.sqrt(R_val)

# Define initial guesses to explore behaviors
initial_guesses = {
    'Standard (x0=12)': 12.0,
    'Close to zero (x0=0.5)': 0.5,
    'Negative (x0=-12)': -12.0
}
```

Python Implementation II

```python

n_iters = 8

# Analytical functions
def g(x): return 0.5 * (x + R_val / x)                # Iteration sequence
def dg(x): return 0.5 * (1 - R_val / x**2)            # Derivative for stability
def f(x): return x**2 - R_val                         # Inferred function (Method 2)
def df(x): return 2 * x                               # Derivative of inferred function

# ---------------------------------------------------------
# 1. Symbolic Verification
# ---------------------------------------------------------
print("--- SymPy Verification (Reverse-Engineering) ---")
x_sym, R_sym = sp.symbols('x R', real=True)
f_sym = x_sym**2 - R_sym
f_prime_sym = sp.diff(f_sym, x_sym)
nr_formula = x_sym - (f_sym / f_prime_sym)
simplified_nr = sp.simplify(nr_formula)
print(f"Standard N-R Formula: x - (x^2 - R)/(2x)")
print(f"Simplified: {simplified_nr}\n")

```

Python Implementation III

```python
# ---------------------------------------------------------
# 2. Numerical Data Generation
# ---------------------------------------------------------
sequences = {}
errors = {}

for name, x0 in initial_guesses.items():
    seq = [x0]
    for _ in range(n_iters):
        seq.append(g(seq[-1]))
    sequences[name] = np.array(seq)

    # Error relative to the root it converges to (+5 or -5)
    target_root = true_root_pos if x0 > 0 else true_root_neg
    errors[name] = np.abs(sequences[name] - target_root)

# ---------------------------------------------------------
# Plot 1: Sequence Convergence (Multi-Path)
# ---------------------------------------------------------
plt.figure(figsize=(10, 6))
colors = ['b', 'orange', 'r']
for (name, seq), color in zip(sequences.items(), colors):
```

Python Implementation IV

```
    plt.plot(range(n_iters + 1), seq, marker='o', linestyle='-', color=color, label=f'{name}')

plt.axhline(y=true_root_pos, color='g', linestyle='--', label=f'+ Root ($\sqrt{{R}}$ = {true_root_pos})')
plt.axhline(y=true_root_neg, color='purple', linestyle='--', label=f'- Root ($-\sqrt{{R}}$ = {true_root_neg})')
plt.title('Convergence of Sequence for Different Initial Guesses')
plt.xlabel('Iteration ($n$)')
plt.ylabel('Value of $x_n$')
plt.xticks(range(n_iters + 1))
plt.grid(True, alpha=0.6)
plt.legend()
plt.tight_layout()
plt.savefig(os.path.join(output_dir, 'plot1_convergence_multi.png'), dpi=300)
plt.close()

# ---------------------------------------------------------
# Plot 2: Error Analysis (Log Scale)
# ---------------------------------------------------------
plt.figure(figsize=(10, 6))
for (name, err), color in zip(errors.items(), colors):
```

Python Implementation V

```
    plt.semilogy(range(n_iters + 1), err, marker='s', linestyle='-', color=color, label=f'{name}')

plt.title('Absolute Error vs Iteration (Log Scale)')
plt.xlabel('Iteration ($n$)')
plt.ylabel('Absolute Error $|x_n - Root|$')
plt.xticks(range(n_iters + 1))
plt.grid(True, which="both", ls="--", alpha=0.6)
plt.legend()
plt.tight_layout()
plt.savefig(os.path.join(output_dir, 'plot2_error_multi.png'), dpi=300)
plt.close()

# ------------------------------------------------------------
# Plot 3: Geometric Interpretation of Newton-Raphson
# ------------------------------------------------------------
x_vals_geom = np.linspace(2, 13, 500)
y_vals_geom = f(x_vals_geom)

plt.figure(figsize=(10, 6))
plt.plot(x_vals_geom, y_vals_geom, label='$f(x) = x^2 - R$', color='black', linewidth=2)
plt.axhline(0, color='black', linewidth=1)
```

Python Implementation VI

```python

seq_std = sequences['Standard (x0=12)']
for i in range(3):
    xn = seq_std[i]
    yn = f(xn)
    xn_next = seq_std[i+1]

    plt.plot(xn, yn, marker='o', color='red')
    plt.vlines(xn, 0, yn, linestyles='dotted', colors='red', alpha=0.7)

    tangent_x = np.array([xn_next, xn])
    tangent_y = df(xn) * (tangent_x - xn) + yn
    plt.plot(tangent_x, tangent_y, color='red', linestyle='--',
             label=f'Step {i+1}: $x_{i}={xn:.2f} \\rightarrow x_{i+1}={xn_next:.2f}$' if i==0 else "")

plt.plot(true_root_pos, 0, marker='*', color='gold', markersize=15, markeredgecolor='black', label='Root $\sqrt{R}$')
plt.title('Geometric Execution of Newton-Raphson (Standard Guess)')
plt.xlabel('$x$')
plt.ylabel('$f(x)$')
plt.legend()
plt.grid(True, alpha=0.3)
```

Python Implementation VII

```python
plt.tight_layout()
plt.savefig(os.path.join(output_dir, 'plot3_geometric_nr.png'), dpi=300)
plt.close()

# ---------------------------------------------------------
# Plot 4, 5, 6: Cobweb Plots for Different Initial Guesses
# ---------------------------------------------------------
def plot_cobweb(x0, name, filename, x_range, y_range):
    x_vals = np.linspace(x_range[0], x_range[1], 1000)
    with np.errstate(divide='ignore'):  # Ignore zero-division warnings locally
        y_vals = g(x_vals)

    plt.figure(figsize=(8, 8))
    plt.plot(x_vals, y_vals, label='$y = g(x)$', color='blue', linewidth=2)
    plt.plot(x_vals, x_vals, label='$y = x$', color='gray', linestyle='--')

    cx, cy = [x0], [0]
    xc = x0
    for _ in range(5):
        yc = g(xc)
        cx.extend([xc, yc])
        cy.extend([yc, yc])
        xc = yc
```

Python Implementation VIII

```python

    plt.plot(cx, cy, color='red', marker='.', linestyle='-', linewidth=1.5, label='Path')

    target_root = true_root_pos if x0 > 0 else true_root_neg
    plt.plot(target_root, target_root, marker='*', color='gold', markersize=15, markeredgecolor='black', label=f'Root = {target_root}')

    plt.title(f'Cobweb Plot: {name}')
    plt.xlabel('$x_n$')
    plt.ylabel('$x_{n+1}$')
    plt.xlim(x_range)
    plt.ylim(y_range)
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.savefig(os.path.join(output_dir, filename), dpi=300)
    plt.close()

# Generate cobwebs
plot_cobweb(12.0, 'Standard Guess (x0=12)', 'plot4_cobweb_std.png', [2, 14], [2, 14])
```

Python Implementation IX

```python
plot_cobweb(0.5, 'Close to zero (x0=0.5)', 'plot5_cobweb_small.png', [0.1, 26], [0, 26])
plot_cobweb(-12.0, 'Negative Guess (x0=-12)', 'plot6_cobweb_neg.png', [-14, -2], [-14, -2])

# ---------------------------------------------------------
# Plot 7: Full Convergence Stability (Derivative Test)
# ---------------------------------------------------------
x_vals_stab = np.linspace(-15, 15, 1000)
x_vals_stab = x_vals_stab[np.abs(x_vals_stab) > 1.5] # Exclude small region around 0
dy_vals = dg(x_vals_stab)

plt.figure(figsize=(10, 6))
plt.plot(x_vals_stab, dy_vals, label="$g'(x)$", color='purple', linewidth=2)
plt.axhline(1, color='red', linestyle=':', alpha=0.5, label='Stability Bound $\pm 1$')
plt.axhline(-1, color='red', linestyle=':', alpha=0.5)
plt.axhline(0, color='black', linewidth=1)

plt.plot(true_root_pos, dg(true_root_pos), marker='o', color='green', markersize=8, label=f"Positive Root (+5)")
```

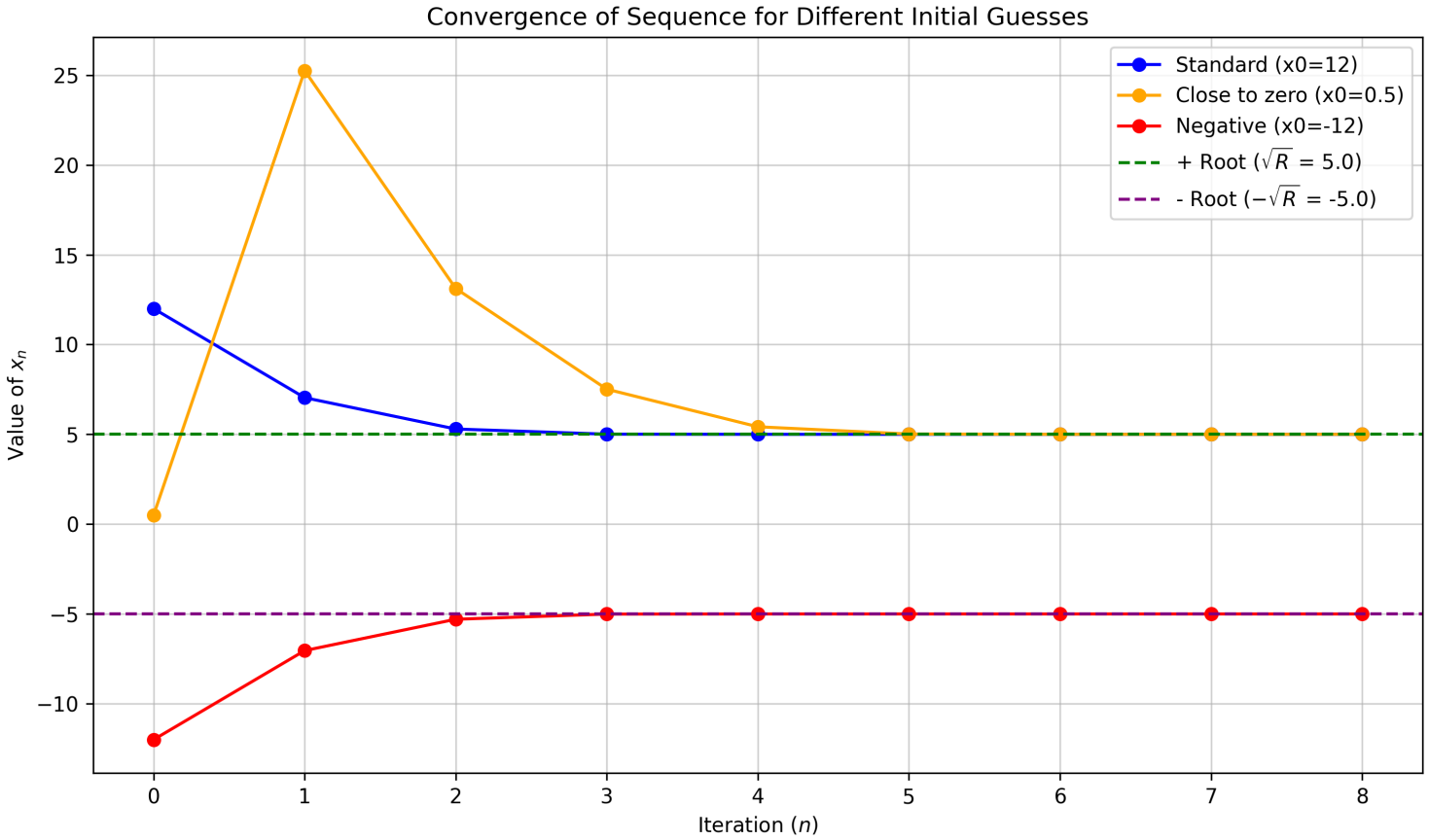
Python Implementation X

```python
plt.plot(true_root_neg, dg(true_root_neg), marker='o', color='blue', markersize=8, label=f"Negative Root (-5)")

plt.fill_between(x_vals_stab, -1, 1, color='green', alpha=0.1, label='Stable Region')
plt.title('Stability Analysis of $g(x)$ over Full Domain')
plt.xlabel('$x$')
plt.ylabel("$g'(x)$")
plt.ylim(-2, 1.5)
plt.legend(loc='lower right')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig(os.path.join(output_dir, 'plot7_stability.png'), dpi=300)
plt.close()

print("All unified plots generated and saved to '../figs/' successfully.")
```

Method 1: Sequence Convergence

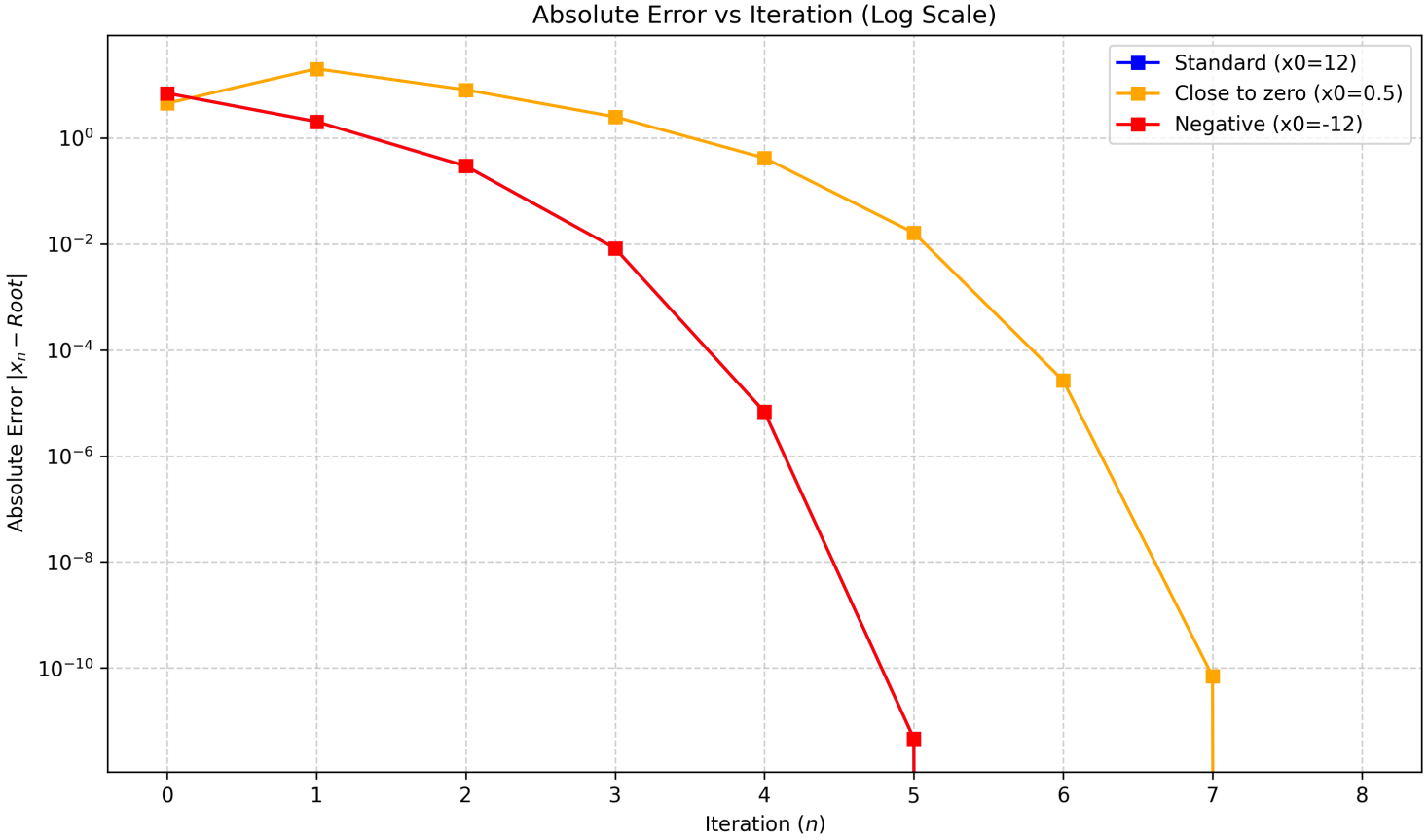
This plot shows the iteration sequence $x_{n+1} = \frac{1}{2}\left(x_n + \frac{25}{x_n}\right)$ for three different initial guesses.

While the standard ($x_0 = 12$) and small ($x_0 = 0.5$) guesses converge to the positive root $\sqrt{25} = 5.0$, the negative guess ($x_0 = -12$) correctly converges to the alternate valid root -5.0.



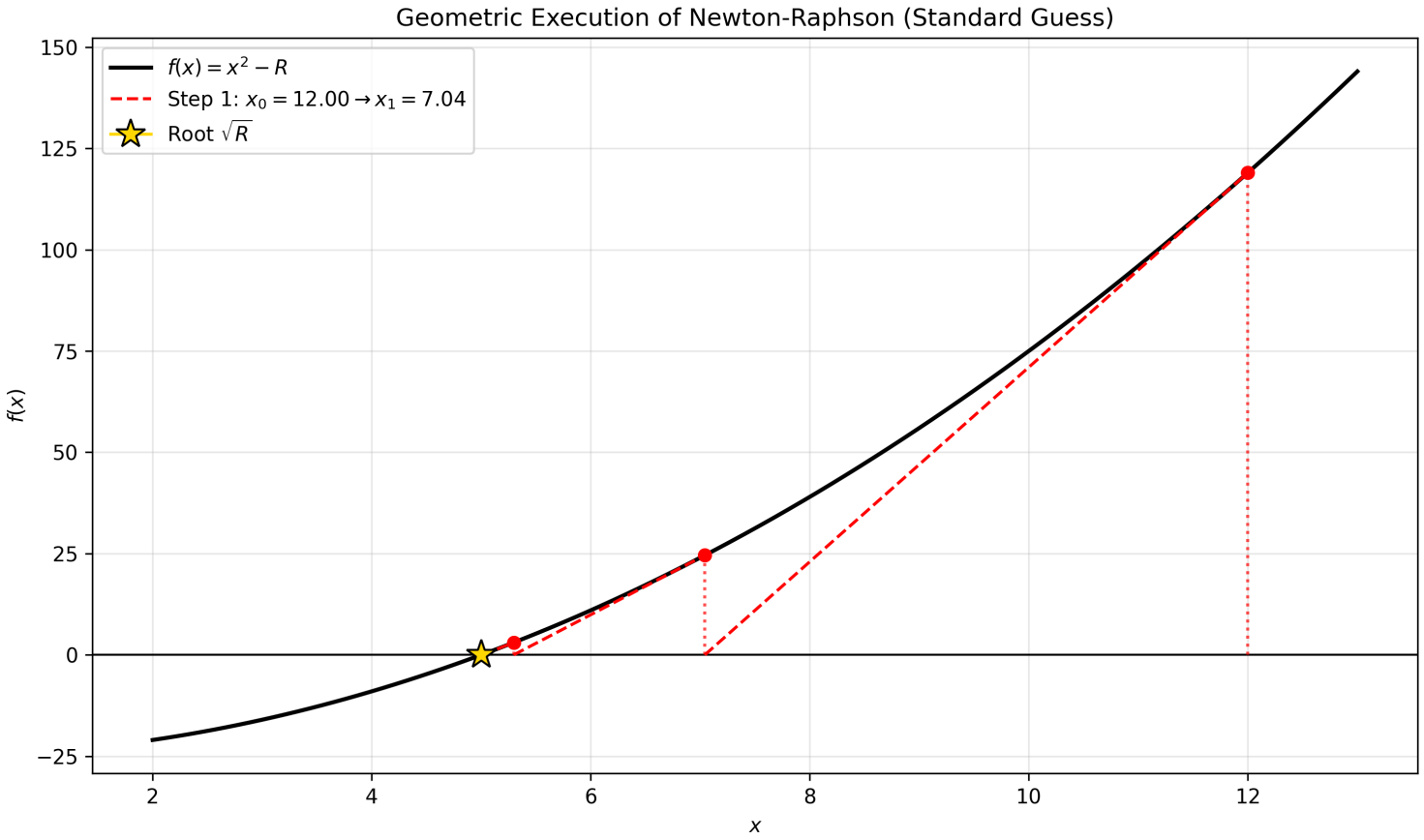
Method 1: Absolute Error Analysis

This semi-log plot visualizes the absolute error over iterations. Despite differing initial trajectories (especially the initial spike for the $x_0 = 0.5$ guess), all paths eventually lock into a steep, parabolic downward curve. This uniform drop-off is the hallmark of **super-linear (quadratic) convergence**.



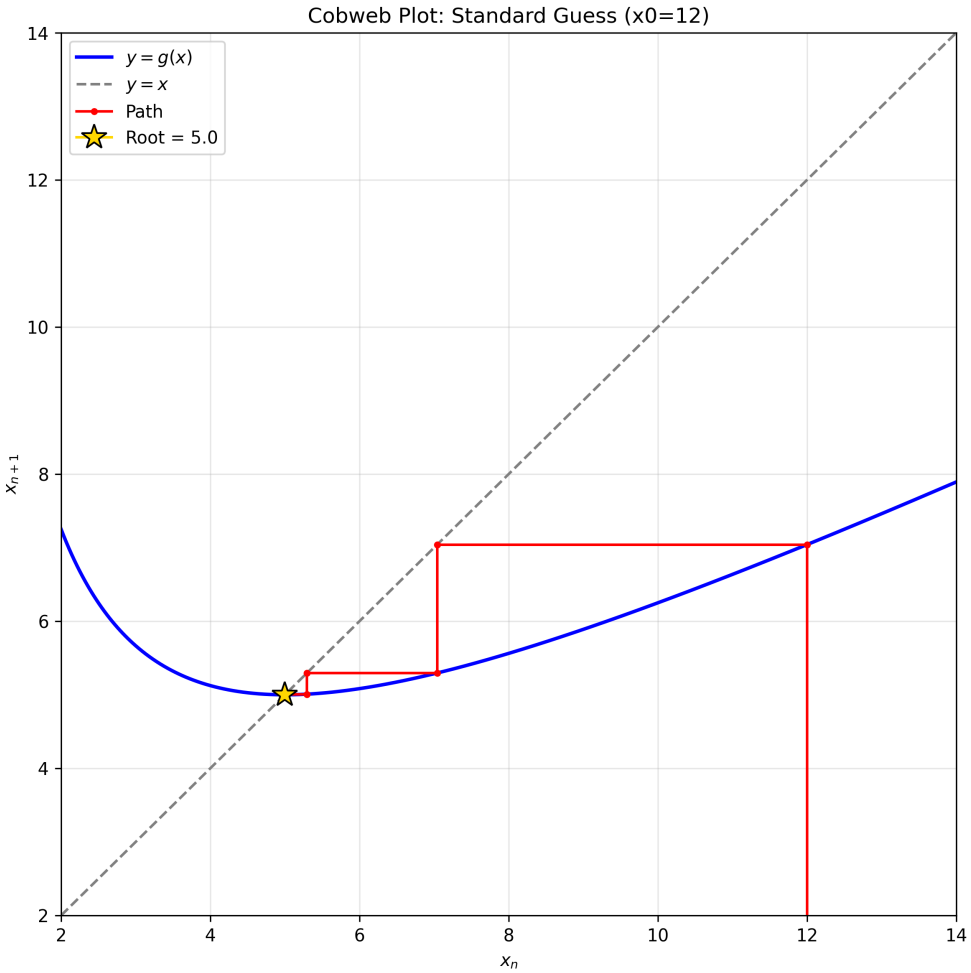
Method 2: Geometric Execution

This figure visualizes the geometric root-finding process on the function $f(x) = x^2 - R$ for a standard initial guess ($x_0 = 12$).
The tangent line physically illustrates the Newton-Raphson relationship $x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$, showing the aggressive mathematical jump toward the true root.



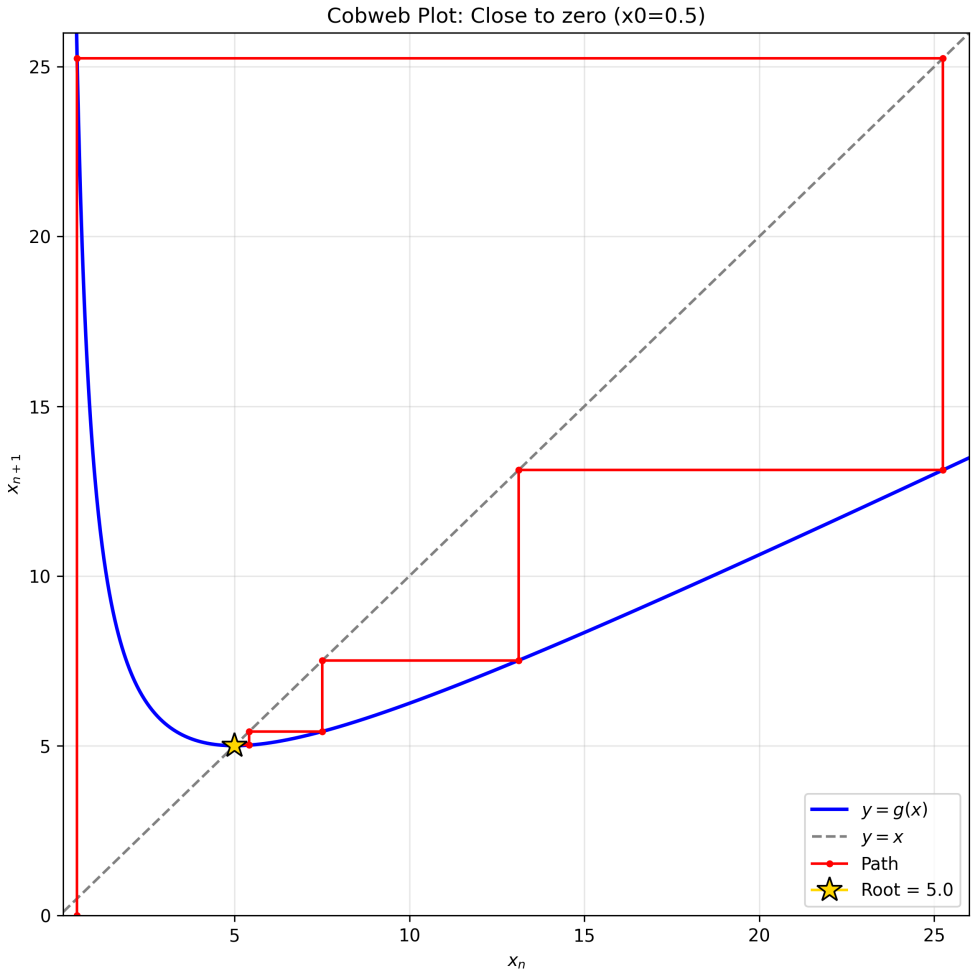
Method 3: Cobweb Plot (Standard Guess)

This cobweb diagram traces the fixed-point iteration for $x_0 = 12$. The sequence updates by bouncing between the curve $y = g(x)$ and the identity line $y = x$, visually funneling directly into the fixed point at $x = 5$.



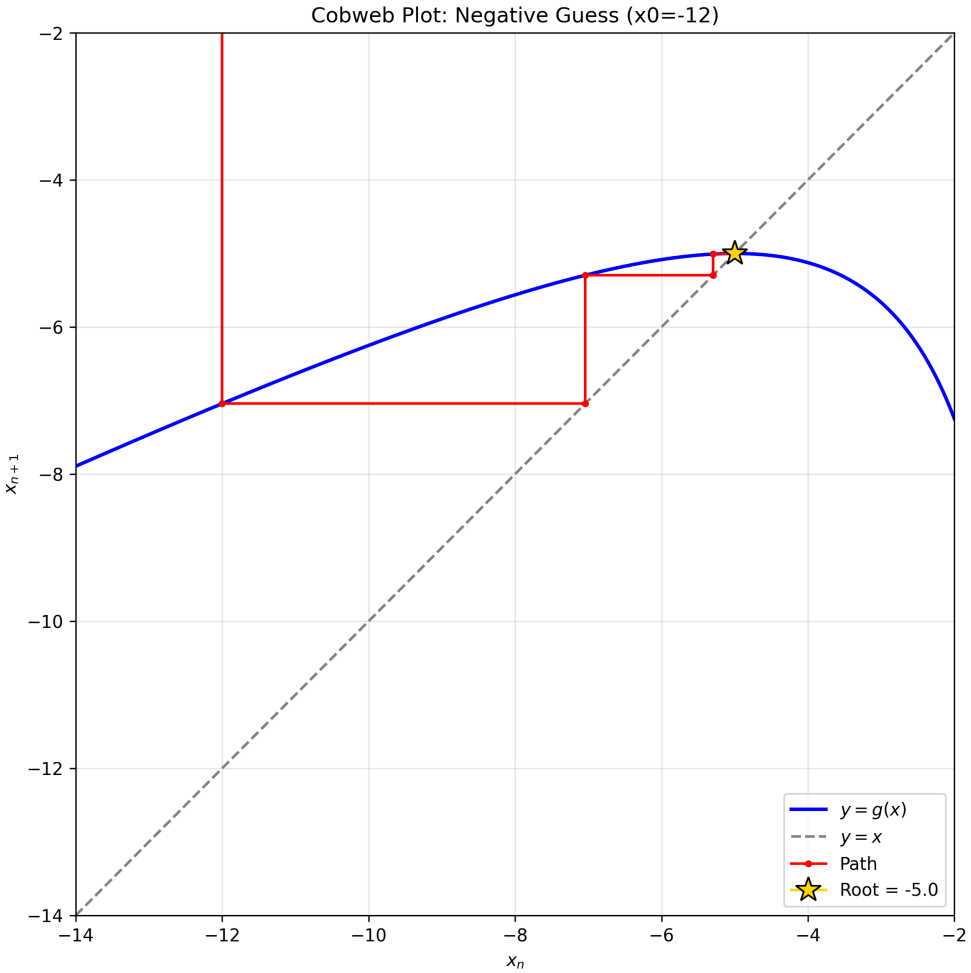
Method 3: Cobweb Plot (Small Guess)

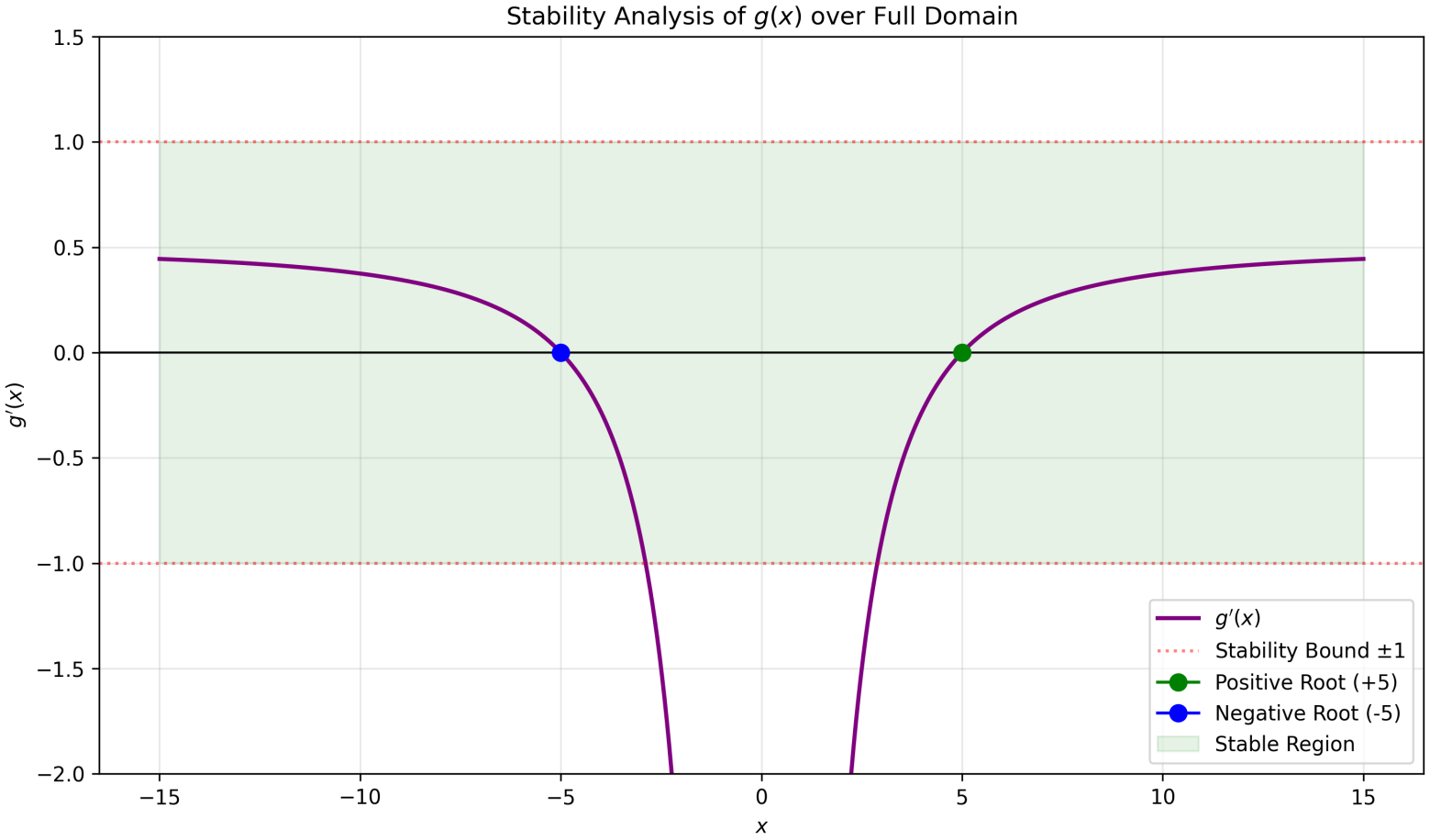
Starting with a guess very close to zero ($x_0 = 0.5$) causes a massive initial jump due to the $\frac{R}{x_n}$ term dominating. However, once the sequence lands to the right of the root, it quickly converges back to $x = 5$.



Method 3: Cobweb Plot (Negative Guess)

Because the function $g(x) = \frac{1}{2}(x + \frac{R}{x})$ is an odd function, initializing with a negative value ($x_0 = -12$) simply mirrors the process in the third quadrant, converging safely to the alternate valid root $-\sqrt{R} = -5$.



Stability & Convergence Bound Analysis

This plot visualizes the derivative $g'(x) = \frac{1}{2}(1 - \frac{R}{x^2})$. The green shaded region marks the stability bounds where $|g'(x)| < 1$. At exactly $x = \pm 5$, $g'(x) = 0$, visually confirming the superlinear (quadratic) convergence at the roots.